

Contents

Lösungen (einfache Variante) — NUR für den Dozenten	1
Epic 1	1
Epic 2	3
Epic 3	6
Epic 4	10
Epic 5	10

Lösungen (einfache Variante) — NUR für den Dozenten

Nicht an Schüler weitergeben, bevor sie es selbst versucht haben.

Diese Datei ist die **Schwester** von [loesungen.md](#). Der Unterschied:

- `loesungen.md` zeigt **kompakte**, **“kotlinige”** Lösungen (when, ausgelagerte Hilfsfunktionen, Elvis-Operator, `minByOrNull`-Ketten). Gut als Referenz für den Dozenten — aber oft **weiter weg** von dem, was ein Kotlin-Anfänger in der 10. Klasse tatsächlich hinschreibt.
- **Diese Datei** zeigt bewusst **geradlinige**, **ausführliche** Lösungen: viel `if/else`, sprechende Variablen, ruhig mal Code doppelt statt clever ausgelagert. So sieht es aus, wenn Schüler es selbst erarbeiten — genau das, worauf ihr beim Review realistisch trefft.

Wofür nutzen? - Als Vergleichsmaßstab beim Review (“das ist völlig ok so, muss nicht kürzer”). - Als Hilfe für Teams, die bei der kompakten Version aussteigen. - Um Schülern zu zeigen: **funktionierend und verständlich schlägt kurz und clever**.

Alle Beispiele gehen von diesen Imports aus:

```
import framework.arena.Action
import framework.arena.Direction
import framework.arena.RobotBrain
import framework.arena.Sensors
import kotlin.math.abs
```

Epic 1

1.1 — Zufällige Bewegung

```
class MeinBot(override val name: String = "Team A - MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val zufallsRichtung = Direction.entries.random()
        return Action.Move(zufallsRichtung)
    }
}
```

Hinweise für den Dozenten: - Bewusst eine eigene Variable `zufallsRichtung` statt alles in eine Zeile — für Anfänger leichter zu lesen und zu debuggen. - `Direction.values().random()` (statt `.entries`) ist genauso richtig; nicht bemängeln.

1.2 — Am Rand bleiben / Zentrum meiden

```
class MeinBot(override val name: String = "Team A - MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
```

```

val meinX = sensors.self.position.x
val meinY = sensors.self.position.y
val mitteX = sensors.arenaWidth / 2
val mitteY = sensors.arenaHeight / 2

// Wie weit bin ich von der Mitte weg?
val abstandX = abs(meinX - mitteX)
val abstandY = abs(meinY - mitteY)

// Stehe ich zu nah an der Mitte?
if (abstandX <= 1 && abstandY <= 1) {
    // Ja -> ich laufe zum näheren waagerechten Rand
    if (meinX < mitteX) {
        return Action.Move(Direction.WEST)
    } else {
        return Action.Move(Direction.EAST)
    }
}

// Nein, weit genug weg -> einfach zufällig bewegen
return Action.Move(Direction.entries.random())
}
}

```

Hinweise für den Dozenten: - Nur die x-Achse wird zum Ausweichen benutzt — völlig ausreichend für die Story. Wer will, ergänzt analog `meinY < mitteY -> NORTH else SOUTH`. - Die vielen Zwischenvariablen (`meinX`, `mitteX`, `abstandX`, ...) sind Absicht: so kann ein Schüler jeden Schritt einzeln nachvollziehen und im Zweifel ausdrucken.

1.3 — Flucht bei niedriger Gesundheit

```

class MeinBot(override val name: String = "Team A - MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        // Gibt es überhaupt einen Gegner?
        if (sensors.others.isEmpty()) {
            return Action.Wait
        }

        // Nächsten Gegner suchen (den mit dem kleinsten Abstand)
        var naechster = sensors.others[0]
        var kleinstAbstand = abs(naechster.position.x - sensors.self.position.x) +
            abs(naechster.position.y - sensors.self.position.y)
        for (gegner in sensors.others) {
            val abstand = abs(gegner.position.x - sensors.self.position.x) +
                abs(gegner.position.y - sensors.self.position.y)
            if (abstand < kleinstAbstand) {
                kleinstAbstand = abstand
                naechster = gegner
            }
        }

        val dx = naechster.position.x - sensors.self.position.x
        val dy = naechster.position.y - sensors.self.position.y
    }
}

```

```

// Wenig HP -> WEG vom Gegner laufen (Vorzeichen umgedreht)
if (sensors.self.health < 20) {
    if (abs(dx) > abs(dy)) {
        if (dx > 0) {
            return Action.Move(Direction.WEST) // Gegner rechts -> ich nach links
        } else {
            return Action.Move(Direction.EAST)
        }
    } else {
        if (dy > 0) {
            return Action.Move(Direction.NORTH) // Gegner unten -> ich nach oben
        } else {
            return Action.Move(Direction.SOUTH)
        }
    }
}

// Genug HP -> ganz normal auf den Gegner ZU laufen
if (abs(dx) > abs(dy)) {
    if (dx > 0) {
        return Action.Move(Direction.EAST)
    } else {
        return Action.Move(Direction.WEST)
    }
} else {
    if (dy > 0) {
        return Action.Move(Direction.SOUTH)
    } else {
        return Action.Move(Direction.NORTH)
    }
}
}
}

```

Hinweise für den Dozenten: - Der nächste Gegner wird hier **mit einer for-Schleife von Hand** gesucht statt mit `minByOrNull`. Das ist länger, aber viele Schüler haben `minByOrNull` noch nicht verinnerlicht — die Schleife ist für sie durchsichtiger. Beide Wege sind richtig. - Flucht- und Verfolgungs-Block sehen fast gleich aus, nur mit vertauschten Richtungen. Bewusst **dupliziert** statt in eine Hilfsfunktion gezogen — so sieht man beim Vergleichen direkt den einzigen Unterschied (das Vorzeichen). - Kritische Review-Stelle: läuft der Bot bei `health < 20` wirklich **weg**? Häufigster Fehler ist genau hier ein verdrehtes Vorzeichen.

Epic 2

2.1 — Dauerfeuer in feste Richtung

```

class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        return Action.Shoot(Direction.EAST)
    }
}

```

Nichts zu erklären — gegen StillstandBot testen lassen, damit Treffer sichtbar werden.

2.2 — Auf Gegner in Sichtlinie zielen

```
class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val meinX = sensors.self.position.x
        val meinY = sensors.self.position.y

        // Alle Gegner durchgehen und den nächsten suchen, der in Linie steht
        var ziel = sensors.self           // Platzhalter, wird gleich ersetzt
        var habeZiel = false
        var kleinsterAbstand = 999999

        for (gegner in sensors.others) {
            val gleicheSpalte = gegner.position.x == meinX
            val gleicheZeile = gegner.position.y == meinY
            if (gleicheSpalte || gleicheZeile) {
                val abstand = abs(gegner.position.x - meinX) + abs(gegner.position.y -
                meinY)
                if (abstand < kleinsterAbstand) {
                    kleinsterAbstand = abstand
                    ziel = gegner
                    habeZiel = true
                }
            }
        }

        // Kein Gegner in Linie -> nicht ins Leere schießen
        if (!habeZiel) {
            return Action.Wait
        }

        // Schussrichtung aus der Position des Ziels ableiten
        if (ziel.position.x == meinX) {
            // gleiche Spalte -> oben oder unten
            if (ziel.position.y < meinY) {
                return Action.Shoot(Direction.NORTH)
            } else {
                return Action.Shoot(Direction.SOUTH)
            }
        } else {
            // gleiche Zeile -> links oder rechts
            if (ziel.position.x > meinX) {
                return Action.Shoot(Direction.EAST)
            } else {
                return Action.Shoot(Direction.WEST)
            }
        }
    }
}
```

Hinweise für den Dozenten: - Statt `filter { }.minByOrNull { }` wird alles in **einer** for-Schleife erledigt: durchgehen, prüfen “in Linie?”, nächsten merken. Länger, aber Schritt für Schritt lesbar. -

Das `habeZiel`-Flag ersetzt hier den Elvis-Operator (`? : return Action.Wait`). Anfänger kommen mit einem Boolean-Flag oft besser klar als mit null-Logik. - `var ziel = sensors.self` ist nur ein Platzhalter, damit die Variable einen Typ hat — genutzt wird sie erst, wenn `habeZiel == true`. Das ist etwas unschön, aber ein sehr typisches Anfänger-Muster; funktioniert korrekt. - Wichtige Review-Stelle: `y < meinY` heißt NORTH (`y` wächst nach unten!).

2.3 — Gegner verfolgen, wenn nicht in Schusslinie

```
class MeinBot(override val name: String = "Team A - MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        if (sensors.others.isEmpty()) {
            return Action.Wait
        }

        val meinX = sensors.self.position.x
        val meinY = sensors.self.position.y

        // Nächsten Gegner suchen
        var naechster = sensors.others[0]
        var kleinstAbstand = abs(naechster.position.x - meinX) + abs(naechster.position.y - meinY)
        for (gegner in sensors.others) {
            val abstand = abs(gegner.position.x - meinX) + abs(gegner.position.y - meinY)
            if (abstand < kleinstAbstand) {
                kleinstAbstand = abstand
                naechster = gegner
            }
        }

        val stehtInLinie = (naechster.position.x == meinX) || (naechster.position.y == meinY)

        if (stehtInLinie) {
            // Schießen (wie in 2.2)
            if (naechster.position.x == meinX) {
                if (naechster.position.y < meinY) {
                    return Action.Shoot(Direction.NORTH)
                } else {
                    return Action.Shoot(Direction.SOUTH)
                }
            } else {
                if (naechster.position.x > meinX) {
                    return Action.Shoot(Direction.EAST)
                } else {
                    return Action.Shoot(Direction.WEST)
                }
            }
        } else {
            // Nicht in Linie -> einen Schritt hinlaufen
            val dx = naechster.position.x - meinX
            val dy = naechster.position.y - meinY
            if (abs(dx) > abs(dy)) {
                if (dx > 0) {
                    return Action.Move(Direction.EAST)
                }
            }
        }
    }
}
```

```

        } else {
            return Action.Move(Direction.WEST)
        }
    } else {
        if (dy > 0) {
            return Action.Move(Direction.SOUTH)
        } else {
            return Action.Move(Direction.NORTH)
        }
    }
}
}
}
}
}
}
}
}

```

Hinweise für den Dozenten: - Das ist der ChaserBot, aber ohne when und ohne Hilfsfunktion — nur if/else. - Schuss-Block ist wortwörtlich aus 2.2 kopiert. Für einen finalen Bot würde man das zusammenfassen; auf Anfänger-Niveau ist Kopieren hier didaktisch ok, weil der Schüler sieht, dass “in Linie = schießen wie vorher”.

Epic 3

3.1 — Zustandsmaschine (Patrouille / Angriff / Flucht)

```

class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val meinX = sensors.self.position.x
        val meinY = sensors.self.position.y

        // 1. Gibt es einen Gegner? Wenn nein -> PATROUILLE
        if (sensors.others.isEmpty()) {
            // patrouillieren = einfach zufällig bewegen
            return Action.Move(Direction.entries.random())
        }

        // Nächsten Gegner suchen
        var naechster = sensors.others[0]
        var kleinstAbstand = abs(naechster.position.x - meinX) + abs(naechster.position.y - meinY)
        for (gegner in sensors.others) {
            val abstand = abs(gegner.position.x - meinX) + abs(gegner.position.y - meinY)
            if (abstand < kleinstAbstand) {
                kleinstAbstand = abstand
                naechster = gegner
            }
        }

        val dx = naechster.position.x - meinX
        val dy = naechster.position.y - meinY

        // 2. Wenig HP? -> FLUCHT (hat Vorrang vor Angriff!)
        if (sensors.self.health < 20) {
            if (abs(dx) > abs(dy)) {
                if (dx > 0) {
                    return Action.Move(Direction.WEST)
                }
            }
        }
    }
}

```

```

        } else {
            return Action.Move(Direction.EAST)
        }
    } else {
        if (dy > 0) {
            return Action.Move(Direction.NORTH)
        } else {
            return Action.Move(Direction.SOUTH)
        }
    }
}

// 3. Sonst -> ANGRIFF (in Linie schießen, sonst hinlaufen)
val stehtInLinie = (dx == 0) || (dy == 0)
if (stehtInLinie) {
    if (dx == 0) {
        if (dy < 0) {
            return Action.Shoot(Direction.NORTH)
        } else {
            return Action.Shoot(Direction.SOUTH)
        }
    } else {
        if (dx > 0) {
            return Action.Shoot(Direction.EAST)
        } else {
            return Action.Shoot(Direction.WEST)
        }
    }
} else {
    if (abs(dx) > abs(dy)) {
        if (dx > 0) {
            return Action.Move(Direction.EAST)
        } else {
            return Action.Move(Direction.WEST)
        }
    } else {
        if (dy > 0) {
            return Action.Move(Direction.SOUTH)
        } else {
            return Action.Move(Direction.NORTH)
        }
    }
}
}
}
}

```

Hinweise für den Dozenten: - Hier bewusst **ohne** enum class BotState. Die drei Zustände sind stattdessen als klar kommentierte Blöcke (*// 1. ...*, *// 2. ...*, *// 3. ...*) umgesetzt. Die Story verlangt “mindestens 3 erkennbare Zustände” — kommentierte if-Blöcke erfüllen das. Wer ein enum benutzt (wie in *loesungen.md*), bekommt gerne Extralob, ist aber nicht Pflicht. - **Zentrale Review-Stelle:** Die Reihenfolge muss stimmen — erst PATROUILLE (kein Gegner), dann FLUCHT (wenig HP), dann ANGRIFF. Wird Angriff vor Flucht geprüft, stirbt der fast tote Bot beim Angreifen. - Der *stehtInLinie*-Check nutzt hier *dx == 0 || dy == 0* statt Positions- vergleich — beides gleichwertig (*dx == 0* heißt gleiches x). - Die Bewegungs-/Schuss-Blöcke sind aus 1.3 und 2.3

übernommen. Genau das ist der Lernpunkt: nichts Neues, nur **sortiert**.

3.2 – Zielpriorisierung (schwächster Gegner zuerst)

```
class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        if (sensors.others.isEmpty()) {
            return Action.Wait
        }

        val meinX = sensors.self.position.x
        val meinY = sensors.self.position.y

        // ZIEL-REGEL: den Gegner mit den wenigsten HP zuerst angreifen
        var ziel = sensors.others[0]
        var wenigsteHp = ziel.health
        for (gegner in sensors.others) {
            if (gegner.health < wenigsteHp) {
                wenigsteHp = gegner.health
                ziel = gegner
            }
        }

        // Ab hier genau wie in 2.3, nur mit `ziel` statt `naechster`
        val dx = ziel.position.x - meinX
        val dy = ziel.position.y - meinY

        val stehtInLinie = (dx == 0) || (dy == 0)
        if (stehtInLinie) {
            if (dx == 0) {
                if (dy < 0) {
                    return Action.Shoot(Direction.NORTH)
                } else {
                    return Action.Shoot(Direction.SOUTH)
                }
            } else {
                if (dx > 0) {
                    return Action.Shoot(Direction.EAST)
                } else {
                    return Action.Shoot(Direction.WEST)
                }
            }
        } else {
            if (abs(dx) > abs(dy)) {
                if (dx > 0) {
                    return Action.Move(Direction.EAST)
                } else {
                    return Action.Move(Direction.WEST)
                }
            } else {
                if (dy > 0) {
                    return Action.Move(Direction.SOUTH)
                }
            }
        }
    }
}
```



```

        } else {
            return Action.Move(Direction.NORTH)
        }
    }
}
}
}
}

```

Hinweise für den Dozenten: - Einziger echter Unterschied zu 2.3: die Auswahl-Schleife sucht gegner.health < wenigsteHp statt kleinsten Abstand. Der Rest ist identisch — genau der Lernpunkt der Story (Kriterium tauschen, Struktur bleibt). - Anderes sinnvolles Kriterium (z.B. wieder nächster Gegner) ebenfalls akzeptieren, solange es klar benannt/kommentiert ist.

3.3 — Kür-Aufgabe

Keine feste Lösung (offen). Als Beispiel eine typische, gut machbare Schüler-Idee: **“Nur schießen, wenn genau EIN Gegner in Linie steht”** (sonst lieber ausweichen, um nicht selbst getroffen zu werden):

```

class MeinBot(override val name: String = "Team A - MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val meinX = sensors.self.position.x
        val meinY = sensors.self.position.y

        // Zählen, wie viele Gegner in Linie stehen, und einen davon merken
        var anzahlInLinie = 0
        var einZiel = sensors.self
        for (gegner in sensors.others) {
            if (gegner.position.x == meinX || gegner.position.y == meinY) {
                anzahlInLinie = anzahlInLinie + 1
                einZiel = gegner
            }
        }

        // Nur schießen, wenn genau einer in Linie steht (sicherer Treffer)
        if (anzahlInLinie == 1) {
            val dx = einZiel.position.x - meinX
            val dy = einZiel.position.y - meinY
            if (dx == 0 && dy < 0) return Action.Shoot(Direction.NORTH)
            if (dx == 0 && dy > 0) return Action.Shoot(Direction.SOUTH)
            if (dy == 0 && dx > 0) return Action.Shoot(Direction.EAST)
            return Action.Shoot(Direction.WEST)
        }

        // Sonst: einfach ausweichen / bewegen
        return Action.Move(Direction.entries.random())
    }
}

```

Hinweis für den Dozenten: Die Idee muss mit Sensors/Action ausdrückbar sein (nur aktuelle Momentaufnahme, keine Zukunft, keine Historie ohne var). Vor dem Start kurz Machbarkeit bestätigen.

Epic 4

4.1 — Unit-Test

```
package bots.teama
```

```
import framework.arena.Action
import framework.arena.Direction
import framework.arena.Position
import framework.arena.RobotState
import framework.arena.Sensors
import kotlin.test.Test
import kotlin.test.assertEquals
```

```
class MeinBotTest {
    @Test
    fun `schießt nach Osten wenn Gegner rechts in gleicher Zeile steht`() {
        // Ich stehe bei (2,5), Gegner bei (7,5) -> gleiche Zeile, rechts von mir
        val ich = RobotState(id = "bot-0", teamName = "Team A", position = Position(2,
        val gegner = RobotState(id = "bot-1", teamName = "Team B", position = Position(
        val sensors = Sensors(
            self = ich,
            others = listOf(gegner),
            arenaWidth = 10,
            arenaHeight = 10,
            tick = 1
        )

        val aktion = MeinBot().decide(sensors)

        assertEquals(Action.Shoot(Direction.EAST), aktion)
    }
}
```

Hinweise für den Dozenten: - Setzt voraus, dass MeinBot auf einen Gegner in gleicher Zeile mit Shoot(EAST) reagiert (z.B. die 2.2- oder 2.3-Lösung). Bei reinen Bewegungs- Bots muss der erwartete Wert entsprechend angepasst werden. - Datei muss unter `src/test/kotlin/bots/teama/` liegen und `package bots.teama` haben, sonst findet Gradle den Test nicht. - `./gradlew test` als Abnahme.

4.2 — Testduell protokollieren

Kein Code — organisatorisch.

Epic 5

Beide Storys organisatorisch, kein Code. Bei 5.1 prüfen, dass teamXBots nur die final gewollte Version enthält, und `./gradlew build` grün ist.